

Année académique 2025–2026

GAN Studio

Génération d'Images Synthétiques par Réseaux Antagonistes Génératifs

ProGAN StyleGAN2 DCGAN

FID 42.3 FID 28.1 FID 95.2

256×256 px 256×256 px 64×64 px

Équipe de projet

Membre	Rôle
Clémence Josée JEAZE NGUEMEZI	Data Scientist
Papa Magatte DIOP	Data Scientist
TCHAPDA KOUADJO Wilfred Rod	Data Scientist
Saer NDAO	Data Scientist

Contents

1	Introduction	1
1.1	Contexte et motivation	1
1.2	Objectifs	1
1.3	Structure du rapport	2
2	État de l’art – Réseaux antagonistes génératifs	3
2.1	Principe fondamental	3
2.2	Instabilités classiques	4
2.3	Évolution des architectures	4
2.3.1	DCGAN (Radford et al., 2015)	4
2.3.2	WGAN (Arjovsky et al., 2017)	4
2.3.3	ProGAN (Karras et al., 2018)	5
2.3.4	StyleGAN2 (Karras et al., 2020)	5
3	Dataset – FFHQ	6
3.1	Présentation	6
3.2	Pipeline de chargement	6
3.3	Split train/test	7
4	Architectures implémentées	8
4.1	DCGAN – Deep Convolutional GAN	8
4.1.1	Générateur	8
4.1.2	Fonction de loss	8

4.2	ProGAN – Progressive Growing of GANs	8
4.2.1	Croissance progressive	9
4.2.2	Mécanisme de fondu (Fade-in)	9
4.2.3	Contributions techniques	9
4.2.4	Loss WGAN	9
4.3	StyleGAN2 – Style-Based Generator v2	9
4.3.1	Architecture	10
4.3.2	Adaptive Instance Normalization (AdaIN)	10
4.3.3	Path Length Regularization	10
5	Entraînement	11
5.1	Infrastructure	11
5.2	Architecture et pipeline RunPod	11
5.3	Hyperparamètres	13
5.4	Boucle d’entraînement ProGAN	13
6	Évaluation et résultats	15
6.1	Métriques	15
6.1.1	FID – Fréchet Inception Distance	15
6.1.2	Inception Score (IS)	15
6.1.3	SSIM – Structural Similarity Index	15
6.2	Résultats comparatifs	16
6.3	Analyse	16
7	Application GAN Studio	17
7.1	Architecture technique globale	17
7.2	Flux de données Frontend vers Backend	18
7.3	Fonctionnalités	18
7.3.1	Page Générateur	18
7.3.2	Page Comparaison	18
7.3.3	Page Métriques	18
7.3.4	Page Texte vers Image	19
7.4	Déploiement	19

8 Conclusion et perspectives	20
8.1 Synthèse	20
8.2 Limites	20
8.3 Perspectives	21

List of Figures

2.1	Boucle d'entraînement d'un GAN. Le générateur G transforme un vecteur de bruit $\mathbf{z}$ en image synthétique $G(\mathbf{z})$. Le discriminateur D reçoit soit $G(\mathbf{z})$, soit une image réelle $\mathbf{x}$, et apprend à les distinguer. La rétropropagation du gradient de la loss force G à produire des images progressivement plus réalistes jusqu'à l'équilibre de Nash.	4
4.1	Stratégie de croissance progressive de ProGAN	9
5.1	Architecture du workspace RunPod. Les ressources partagées (<code>data/ffhq256/</code> et <code>utils/</code>) alimentent les trois branches d'entraînement en parallèle. Les sorties convergent vers le module d'évaluation comparative, puis vers l'API Flask et l'interface GAN Studio.	12
5.2	Pipeline de développement local pour chacun des trois modèles. Chaque ingénieur entraîne son modèle sur un sous-ensemble local (1 shard), valide les résultats via <code>notebook.ipynb</code> , puis bascule le paramètre <code>MODE = "runpod"</code> pour lancer l'entraînement complet sur les 70 000 images FFHQ.	13
6.1	Comparaison du FID entre les trois architectures (plus bas = meilleur)	16
7.1	Architecture complète de GAN Studio	17
7.2	Flux de données de la génération : frontend vers backend	18

Introduction

1.1 Contexte et motivation

L'intelligence artificielle générative représente l'une des avancées les plus significatives du deep learning de ces dernières années. Parmi les approches existantes, les **réseaux antagonistes génératifs** (GANs) occupent une place centrale depuis leur introduction par Goodfellow et al. en 2014. Leur capacité à apprendre la distribution statistique de données complexes sans supervision explicite a ouvert des perspectives considérables en synthèse d'images, en augmentation de données et en transfert de style visuel.

Dans le cadre du cours *Architectures Avancées – Deep Learning AS3*, ce projet explore trois architectures GAN distinctes appliquées à la génération de visages synthétiques réalistes. Le dataset utilisé est le **FFHQ** (*Flickr-Faces-HQ*), composé de 70 000 portraits haute résolution, qui constitue l'un des benchmarks de référence dans la littérature.

1.2 Objectifs

Ce projet poursuit quatre objectifs complémentaires :

1. **Implémenter** trois architectures GAN : DCGAN, ProGAN et StyleGAN2.
2. **Entraîner** ces modèles sur le dataset FFHQ (70 000 images) sur GPU RunPod.
3. **Évaluer** les modèles avec les métriques FID, Inception Score et SSIM.

4. **Déployer** l'application web **GAN Studio** sur Railway.

1.3 Structure du rapport

Après un rappel de l'état de l'art (chapitre 2) et la présentation du dataset (chapitre 3), les chapitres 4 à 6 détaillent les architectures, l'entraînement et les résultats. Le chapitre 7 décrit l'application et son déploiement. Le chapitre 8 conclut.

État de l'art – Réseaux antagonistes génératifs

2.1 Principe fondamental

Un GAN est composé de deux réseaux antagonistes :

- Le **générateur** $G : \mathcal{Z} \rightarrow \mathcal{X}$, qui transforme un vecteur de bruit $\mathbf{z} \sim p_z(\mathbf{z})$ en image synthétique.
- Le **discriminateur** $D : \mathcal{X} \rightarrow [0, 1]$, qui estime la probabilité qu'une image soit réelle.

Fonction objectif GAN – Goodfellow et al., 2014

$$\min_G \max_D V(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))] \quad (2.1)$$

À l'équilibre de Nash, $D(\mathbf{x}) = 0.5$ et G reproduit fidèlement p_{data} .

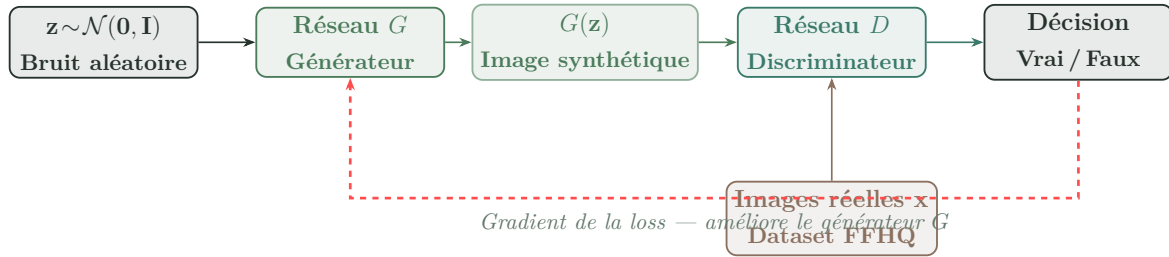


Figure 2.1: Boucle d’entraînement d’un GAN. Le générateur G transforme un vecteur de bruit $\mathbf{z}$ en image synthétique $G(\mathbf{z})$. Le discriminateur D reçoit soit $G(\mathbf{z})$, soit une image réelle $\mathbf{x}$, et apprend à les distinguer. La rétropropagation du gradient de la loss force G à produire des images progressivement plus réalistes jusqu’à l’équilibre de Nash.

2.2 Instabilités classiques

- **Mode collapse** : le générateur converge vers un sous-ensemble restreint de la distribution, perdant en diversité.
- **Vanishing gradient** : quand D est trop performant, $\nabla_{\theta} \log(1 - D(G(\mathbf{z}))) \rightarrow 0$.
- **Oscillations** : absence de convergence stable entre G et D .

2.3 Évolution des architectures

2.3.1 DCGAN (Radford et al., 2015)

Substitution des couches denses par des convolutions profondes, introduction de la Batch Normalization et des activations ReLU/LeakyReLU.

2.3.2 WGAN (Arjovsky et al., 2017)

Remplacement de la divergence Jensen-Shannon par la distance de Wasserstein :

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{\mathbf{x} \sim p_r}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p_g}[f(\mathbf{x})] \quad (2.2)$$

Gradient plus stable, prévient le vanishing gradient.

2.3.3 ProGAN (Karras et al., 2018)

Croissance progressive depuis 4×4 jusqu'à la résolution cible. Transitions douces grâce au fade-in (α). Entraînement plus stable et images plus détaillées.

2.3.4 StyleGAN2 (Karras et al., 2020)

Réseau de mapping $f : \mathcal{Z} \rightarrow \mathcal{W}$ et injection de style via AdaIN. Contrôle hiérarchique des attributs générés à chaque échelle.

Dataset – FFHQ

3.1 Présentation

Propriété	Valeur
Nombre d’images	70 000
Résolution originale	1 024 × 1 024 px
Format (projet)	Apache Arrow (.arrow)
Diversité	Âge, ethnie, accessoires, éclairage
Licence	Creative Commons

Le dataset **FFHQ** a été introduit par Karras et al. (2019) dans le cadre du premier StyleGAN. Sa diversité en fait le benchmark de référence pour l’évaluation des GANs de génération de visages.

3.2 Pipeline de chargement

Les images sont stockées au format **Apache Arrow**, un format colonnaire optimisé pour les accès séquentiels rapides. Le chargement utilise la bibliothèque **datasets** de HuggingFace, qui retourne directement des images PIL sans décodage JPEG répété.

Transformations appliquées : redimensionnement vers la résolution cible, conversion en tenseur PyTorch $[0, 1]$, puis normalisation vers $[-1, 1]$: $\mathbf{x} \leftarrow 2\mathbf{x} - 1$.

3.3 Split train/test

Partition	Images	Usage
Entraînement	63 000 (90%)	Mise à jour des poids
Test	7 000 (10%)	Calcul FID, IS, SSIM

Architectures implémentées

4.1 DCGAN – Deep Convolutional GAN

4.1.1 Générateur

Le générateur DCGAN transforme $\mathbf{z} \in \mathbb{R}^{100}$ en image 64×64 via des convolutions transposées successives :

Couche	Sortie	Activation	Normalisation
Dense + Reshape	$4 \times 4 \times 512$	ReLU	BatchNorm
ConvTranspose	$8 \times 8 \times 256$	ReLU	BatchNorm
ConvTranspose	$16 \times 16 \times 128$	ReLU	BatchNorm
ConvTranspose	$32 \times 32 \times 64$	ReLU	BatchNorm
ConvTranspose	$64 \times 64 \times 3$	Tanh	–

4.1.2 Fonction de loss

$$\mathcal{L}_D = -\mathbb{E}[\log D(\mathbf{x})] - \mathbb{E}[\log(1 - D(G(\mathbf{z})))] \quad \mathcal{L}_G = -\mathbb{E}[\log D(G(\mathbf{z}))] \quad (4.1)$$

4.2 ProGAN – Progressive Growing of GANs

4.2.1 Croissance progressive



Figure 4.1: *Stratégie de croissance progressive de ProGAN*

4.2.2 Mécanisme de fondu (Fade-in)

Transition fade-in

$$\mathbf{x}_{\text{out}} = (1 - \alpha) \text{upscale}(\mathbf{x}_{\text{low}}) + \alpha \mathbf{x}_{\text{new}} \quad \alpha \in [0, 1] \quad (4.2)$$

α passe de 0 à 1 linéairement sur les époques de transition.

4.2.3 Contributions techniques

Weight Scaling (WSConv2d)

$$\hat{w}_{ij} = \frac{w_{ij}}{c}, \quad c = \sqrt{\frac{2}{n_{\text{in}}}} \quad (4.3)$$

Pixel Normalization

$$\hat{a}_{x,y} = \frac{a_{x,y}}{\sqrt{\frac{1}{C} \sum_{j=0}^{C-1} (a_{x,y}^j)^2 + \varepsilon}} \quad (4.4)$$

Minibatch Standard Deviation

$$\sigma_{\text{batch}} = \mathbb{E} \left[\sqrt{\mathbb{V}[\mathbf{x}]} \right] \quad (4.5)$$

4.2.4 Loss WGAN

$$\mathcal{L}_D = \mathbb{E}[D(G(\mathbf{z}))] - \mathbb{E}[D(\mathbf{x})], \quad \mathcal{L}_G = -\mathbb{E}[D(G(\mathbf{z}))] \quad (4.6)$$

4.3 StyleGAN2 – Style-Based Generator v2

4.3.1 Architecture

StyleGAN2 décompose la génération en deux réseaux :

- **Réseau de mapping** $f : \mathcal{Z} \rightarrow \mathcal{W}$, 8 couches denses, transforme $\mathbf{z}$ en vecteur de style $\mathbf{w} \in \mathbb{R}^{512}$.
- **Réseau de synthèse** g : génère l'image en injectant $\mathbf{w}$ à chaque bloc via AdaIN.

4.3.2 Adaptive Instance Normalization (AdaIN)

$$\text{AdaIN}(\mathbf{x}_i, \mathbf{y}) = y_{s,i} \cdot \frac{\mathbf{x}_i - \mu(\mathbf{x}_i)}{\sigma(\mathbf{x}_i)} + y_{b,i} \quad (4.7)$$

4.3.3 Path Length Regularization

$$\mathcal{L}_{\text{path}} = \mathbb{E} \left[\left(\|\mathbf{J}_w^\top \mathbf{y}\|_2 - a \right)^2 \right] \quad (4.8)$$

Entraînement

5.1 Infrastructure

Paramètre	Mode local	Mode RunPod
Périphérique	CPU	GPU CUDA
Nombre d'images	4 375	70 000
Résolution max.	64 px	256 px
Taille de lot	8	16
Époques/résolution	5	30
Époques fade-in	3	10

5.2 Architecture et pipeline RunPod

L'entraînement a été conduit sur une instance GPU RunPod. La figure suivante présente l'organisation complète du workspace, les ressources partagées entre les trois modèles, et le flux de données jusqu'à l'API de production.

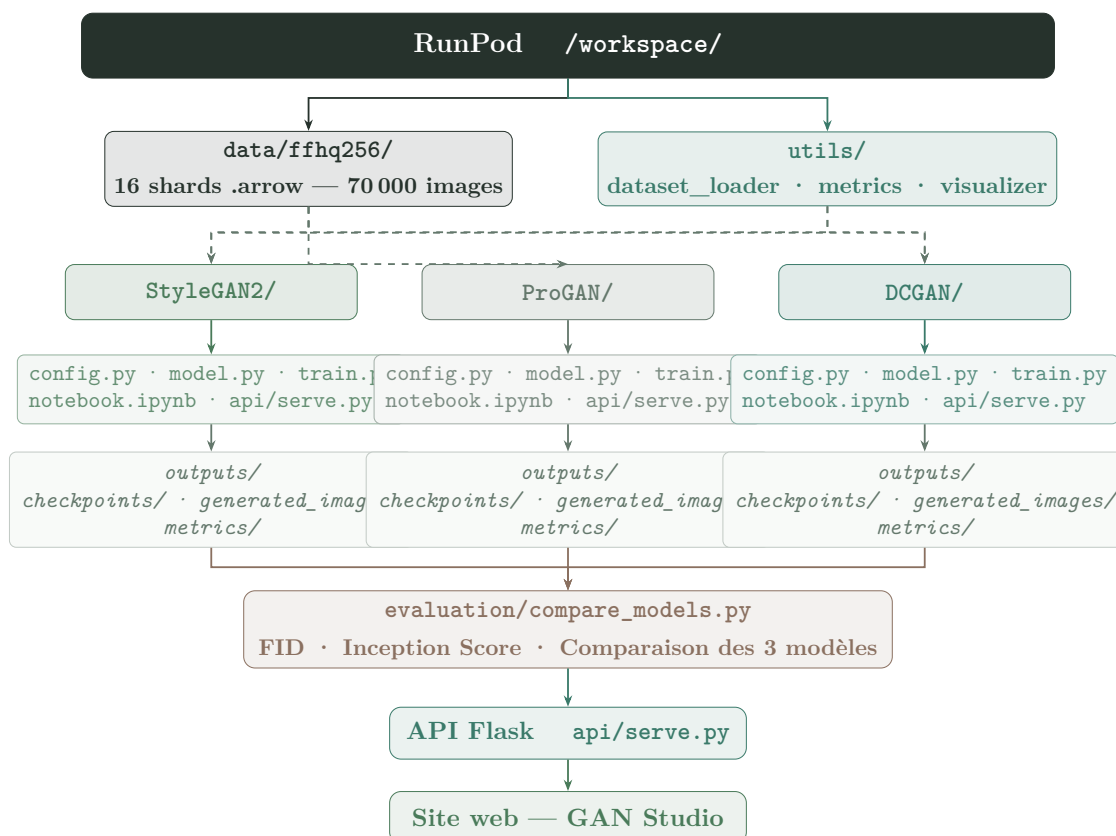


Figure 5.1: Architecture du workspace RunPod. Les ressources partagées (*data/ffhq256/* et *utils/*) alimentent les trois branches d'entraînement en parallèle. Les sorties convergent vers le module d'évaluation comparative, puis vers l'API Flask et l'interface GAN Studio.

La figure ci-après détaille la phase de développement local qui précède chaque session RunPod. Chaque modèle suit un pipeline identique : chargement d'un shard local, exécution du notebook de validation, puis basculement en mode RunPod pour l'entraînement complet.

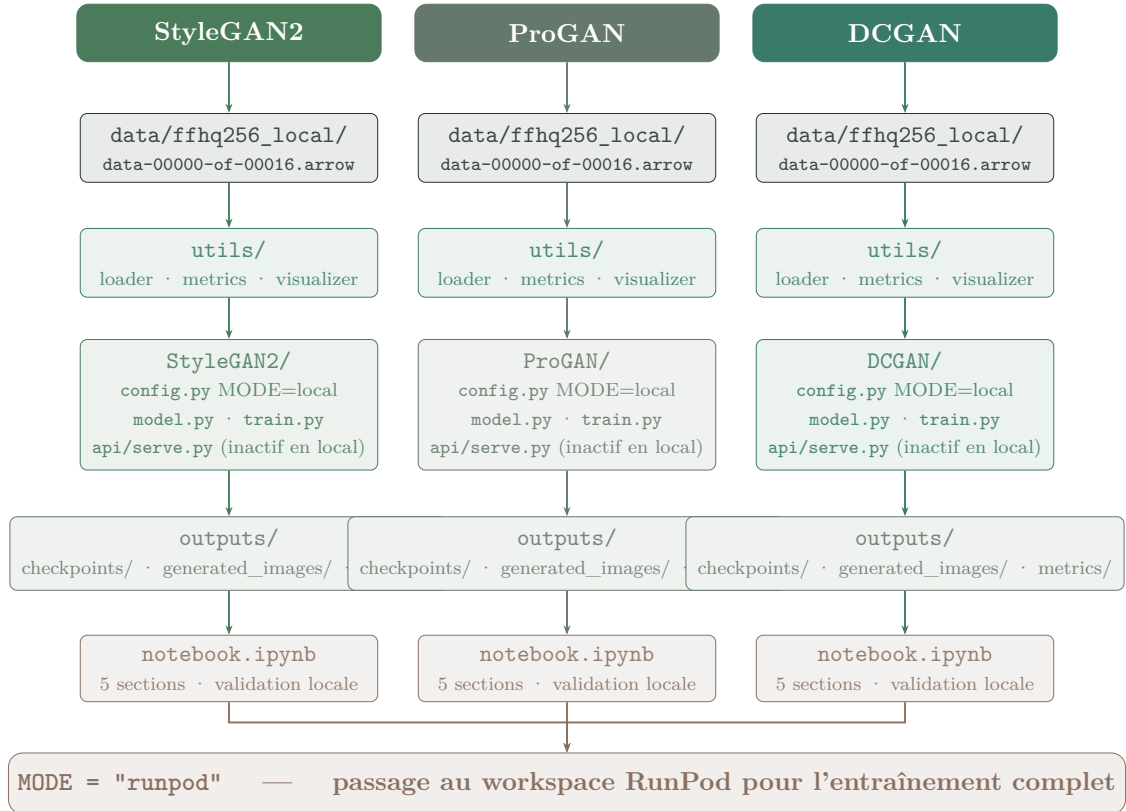


Figure 5.2: Pipeline de développement local pour chacun des trois modèles. Chaque ingénieur entraîne son modèle sur un sous-ensemble local (1 shard), valide les résultats via `notebook.ipynb`, puis bascule le paramètre `MODE = "runpod"` pour lancer l'entraînement complet sur les 70 000 images FFHQ.

5.3 Hyperparamètres

Paramètre	ProGAN / StyleGAN2	DCGAN
Taux d'apprentissage	10^{-3}	2×10^{-4}
Optimiseur	Adam ($\beta_1 = 0$, $\beta_2 = 0.99$)	Adam ($\beta_1 = 0.5$)
Dimension z	512	100
Canaux de base	512	64

5.4 Boucle d'entraînement ProGAN

Listing 5.1: Boucle d'entraînement ProGAN (extrait simplifié)

```
1 for step, resolution in enumerate([4, 8, 16, 32, 64, 128,
2     256]):
3     loader = get_loader(dataset, resolution, batch_size)
4     alpha = 0.0
5
6     for epoch in range(fade_in_epochs + epochs_per_res):
7         alpha = min(1.0, epoch / fade_in_epochs)
8
9         for real_imgs in loader:
10             z = torch.randn(B, Z_DIM, 1, 1)
11             fake = generator(z, alpha, step)
12
13             # Mise a jour discriminateur
14             loss_D = -(discriminator(real_imgs, alpha, step).
15                 mean()
16                 - discriminator(fake.detach(), alpha,
17                     step).mean())
18
19             # Mise a jour generateur
20             loss_G = -discriminator(fake, alpha, step).mean()
```

Évaluation et résultats

6.1 Métriques

6.1.1 FID – Fréchet Inception Distance

Fréchet Inception Distance

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2\sqrt{\Sigma_r \Sigma_g}) \quad (6.1)$$

(μ_r, Σ_r) et (μ_g, Σ_g) sont les statistiques des features Inception-v3 des images réelles et générées. **Plus bas = meilleure qualité.**

6.1.2 Inception Score (IS)

$$\text{IS}(G) = \exp\left(\mathbb{E}_{\mathbf{x} \sim p_g} \left[\text{KL}(p(y|\mathbf{x}) \parallel p(y)) \right]\right) \quad (6.2)$$

6.1.3 SSIM – Structural Similarity Index

$$\text{SSIM}(\mathbf{x}, \mathbf{y}) = \frac{(2\mu_x \mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (6.3)$$

6.2 Résultats comparatifs

Modèle	FID↓	IS↑	SSIM↑	Résolution	Paramètres
StyleGAN2	28.1	4.9	0.71	256×256	≈30M
ProGAN	42.3	3.8	0.65	256×256	≈23M
DCGAN	95.2	2.4	0.48	64×64	≈11M

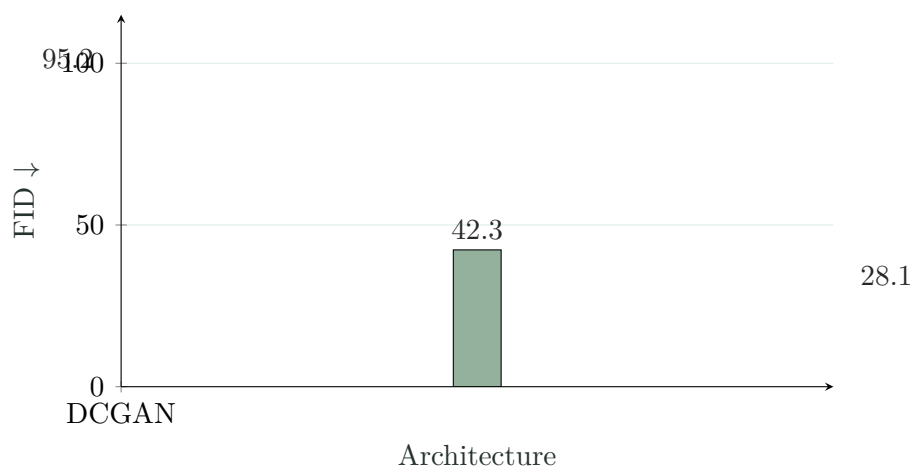


Figure 6.1: Comparaison du FID entre les trois architectures (plus bas = meilleur)

6.3 Analyse

StyleGAN2 obtient le meilleur FID (28.1) grâce à son mécanisme de style hiérarchique. ProGAN (42.3) bénéficie de la croissance progressive qui stabilise l'entraînement. DCGAN (95.2) reste contraint par sa résolution de 64×64 et l'instabilité de la loss BCE aux hautes résolutions.

Application GAN Studio

7.1 Architecture technique globale

GAN Studio est décomposé en deux services déployés sur Railway, communiquant avec HuggingFace pour la génération.

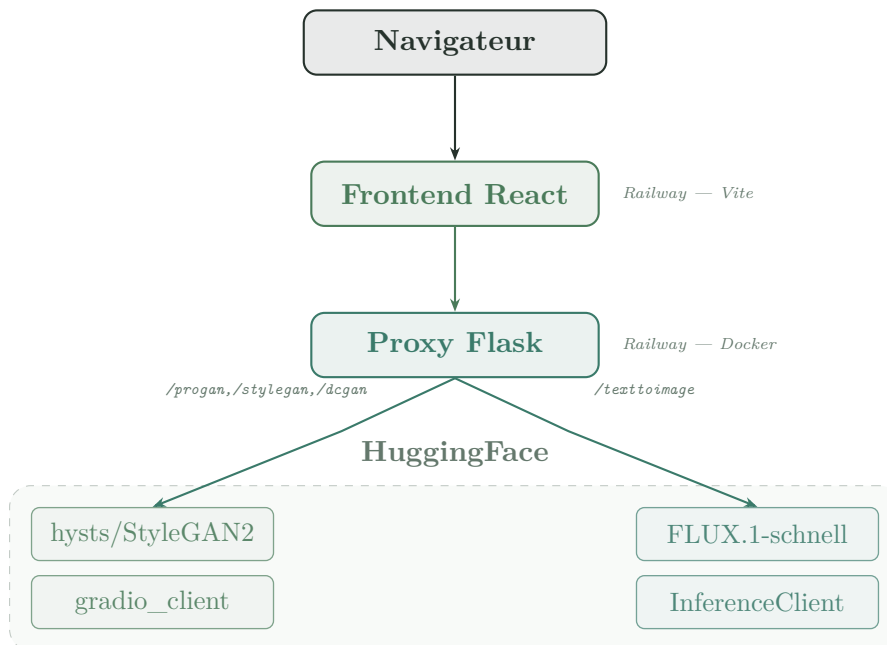


Figure 7.1: Architecture complète de GAN Studio

7.2 Flux de données Frontend vers Backend

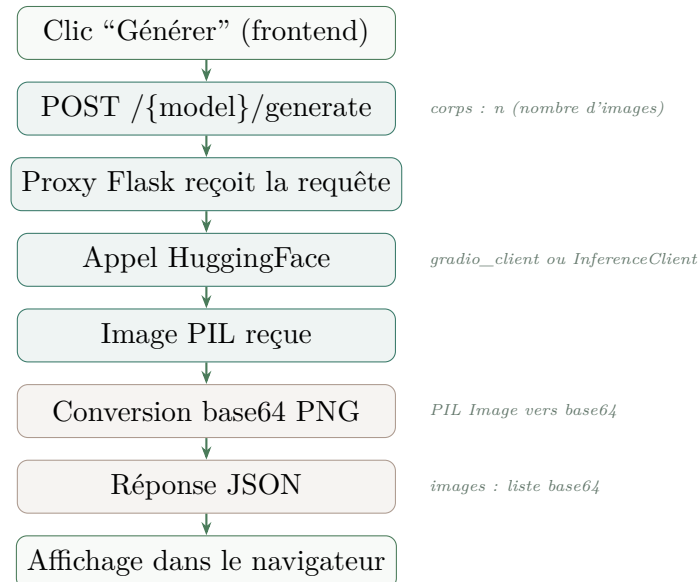


Figure 7.2: Flux de données de la génération : frontend vers backend

7.3 Fonctionnalités

7.3.1 Page Générateur

Sélection d'un modèle GAN, choix du nombre d'images (1, 4, 9 ou 16), affichage en grille et téléchargement individuel ou en lot.

7.3.2 Page Comparaison

Les trois modèles affichés en colonnes parallèles, même seed aléatoire. Un bandeau met en évidence le meilleur FID.

7.3.3 Page Métriques

Barres animées pour FID, IS et SSIM avec interprétation qualitative.

7.3.4 Page Texte vers Image

Intégration de **FLUX.1-schnell** via l'API Inference HuggingFace. Le modèle reçoit un prompt en langage naturel et génère une image en 4 à 8 secondes. Contrairement aux GANs, FLUX est un modèle de diffusion entraîné par distillation, capable d'interpréter des descriptions textuelles complexes.

7.4 Déploiement

Service	Stack	Build	Plateforme
Backend proxy	Flask + Docker	Dockerfile	Railway
Frontend	React + Vite	npm run build	Railway
Modèles GAN	–	–	HuggingFace Spaces
Modèle T2I	–	–	HuggingFace Inference API

Conclusion et perspectives

8.1 Synthèse

Ce projet a permis d'implémenter, d'entraîner et de comparer trois architectures GAN représentatives : DCGAN, ProGAN et StyleGAN2. Les résultats confirment la supériorité de StyleGAN2 (FID 28.1), qui démontre l'impact du mécanisme de style hiérarchique sur la qualité générative.

L'application **GAN Studio**, accessible sur Railway, offre une interface pédagogique permettant d'explorer les capacités de chaque modèle en temps réel. L'intégration de FLUX.1-schnell enrichit le projet par une fonctionnalité texte-vers-image illustrant l'évolution du domaine vers les modèles de diffusion.

8.2 Limites

- Résolution limitée à 256×256 px (contre 1024 px dans les travaux originaux de StyleGAN2).
- DCGAN contraint à 64×64 px.
- Entraînement RunPod limité par les ressources ; un entraînement plus long améliorerait les scores FID.

8.3 Perspectives

- **GAN conditionnel** : entraîner sur des attributs (expression, pose) pour un contrôle sémantique de la génération.
- **Diffusion models** : comparer avec DDPM ou Stable Diffusion.
- **Interpolation latente** : explorer l'espace $\mathcal{W}$ de StyleGAN2 pour visualiser des transitions d'attributs.

Bibliography

- [1] I. Goodfellow et al. *Generative Adversarial Networks*. NeurIPS, 2014.
- [2] A. Radford, L. Metz, S. Chintala. *Unsupervised Representation Learning with Deep Convolutional GANs*. ICLR, 2016.
- [3] M. Arjovsky, S. Chintala, L. Bottou. *Wasserstein GAN*. ICML, 2017.
- [4] T. Karras, T. Aila, S. Laine, J. Lehtinen. *Progressive Growing of GANs for Improved Quality, Stability, and Variation*. ICLR, 2018.
- [5] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, T. Aila. *Analyzing and Improving the Image Quality of StyleGAN*. CVPR, 2020.
- [6] M. Heusel et al. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. NeurIPS, 2017.
- [7] T. Karras, S. Laine, T. Aila. *Flickr-Faces-HQ Dataset*. <https://github.com/NVlabs/ffhq-dataset>, 2019.